/connectors

Connector management: /connectors list [–verbose] [–category] | claude [query] [–category] [–refresh] | add <id|source> [–force] | remove | enable | disable | connect | disconnect | auth | test | stores [–check] | marketplace show | help

Overview

`/connectors` manages **connectors**: named, user-facing integrations (Google Drive, Slack, GitHub, Git, Postgres, Puppeteer, and the wider community catalogue) that reach an external system through one or more **MCP servers**. A connector is a catalogue entry carrying a display name, a category, an authentication requirement, and one or more MCP servers; the MCP server remains the transport ragent already speaks (`McpClient`), so the connector system is a management and catalogue layer over machinery that already exists.

A bare `/connectors`, `/connectors help`, and any unrecognised subcommand all print the same usage block and create or modify no files.

The same operations are available from a shell via the `ragent connectors <sub>` CLI parity surface; the two surfaces share one help text and one set of command handlers.

The system is specified in `specs/connectors/SPEC.md` (FR-001..FR-041); the manual test plan is `specs/connectors/TESTPLAN.md`.

Syntax

```
/connectors list [--verbose] [--category <name>]   # installed connectors + state
/connectors claude [query] [--category <name>] [--refresh]  # interactive catalogue browser
/connectors add <id|source> [--force]              # install (recorded enabled)
/connectors remove <id>                            # uninstall (refused while enabled)
/connectors enable <id>                            # enable + connect now
/connectors disable <id>                           # disable + disconnect
/connectors connect <id>                           # connect an enabled connector
/connectors disconnect <id>                        # disconnect, leave enabled
/connectors auth <id>                              # manage credential / report auth state
/connectors test <id>                              # isolated connect-and-invoke harness
/connectors stores [--check]                       # report catalogue endpoints + provenance
/connectors marketplace show                        # list configured marketplaces
/connectors marketplace claude list|add|remove      # manage added marketplaces
/connectors help                                   # usage block
```

Options / Subcommands

Form	Description
`/connectors list [--verbose] [--category <name>]`	One row per installed connector showing state, auth state, category, and bridged server/tool counts, sorted alphabetically by connector id (case-insensitive). `--verbose` adds detail. `--category` restricts the rows, which keep their alphabetical order; `ALL` (any case) or a blank value clears the filter; an unknown category is refused with an `[err]` row and changes no state.

Form	Description
`/connectors claude [query] [--category <name>] [--refresh]`	Open the interactive Claude connector-catalogue browser, wearing the same design language as `/plugins claude`: a calm slate accent frame and a structured title line (catalogue name badge, visible/total counts, the active `search`/`category`/`marketplace` filters, plus a `loading details...` hint while the manifest-description enrichment runs); a left column split into a `Categories` navigator above a `Marketplace` navigator, each under a reversed-video title band and each row labelled `Name (N)`; a fixed dark-on-light column-heading row (`Id`, `Category`, `Marketplace`, `Description`) pinned above a zebra-banded scrollable result list with the block cursor on the highlighted row (each row shows the connector id, its category, the marketplace it came from, and its description running to the panel's right edge, with a bold green `*` in a one-column lane for an already-installed connector); a centred footer of key hints; and a bottom `search:` box with a clickable `[x] clear` button. The category navigator lists `ALL`, an `Installed` pseudo-category (the connectors already in the store scan), then the catalogue's categories. The optional `query` pre-fills the search field, `--category <name>` pre-selects the category filter (an unknown category is refused in the footer and no row is hidden), and `--refresh` bypasses the catalogue cache. The panel merges the base catalogue endpoint and every marketplace added with `/connectors marketplace claude add`, and fills each entry's blank description from its own repository manifest off the event loop. Three panes are focusable - results, then the two navigators - and `Tab`/`BackTab` cycle through them with only the focused pane's cursor filled; `Up`/`Down` over a navigator apply the highlighted filter immediately, `ENTER` over a navigator applies it and returns focus to the results, and `c` clears the focused navigator's filter back to `ALL`. Clicking a row in either navigator applies it, the wheel scrolls the navigator, and clicking `[x] clear` wipes the query.

Form	Description
`/connectors add <id|source> [--force]`	Install a connector from a catalogue id, a local directory holding a connector manifest, a local `.zip`/`.tar.gz`, or an `https://` URL naming a package. A fresh install is recorded **enabled**; run `/connectors enable <id>` to connect its servers now in a running session. A duplicate id is refused unless `--force` is given.
`/connectors remove <id>`	Uninstall a connector. Refused while the connector is enabled.
`/connectors enable <id>`	Enable a connector and connect its servers now.
`/connectors disable <id>`	Disable a connector and disconnect its servers.
`/connectors connect <id>`	Connect an enabled connector's servers without a session restart.
`/connectors disconnect <id>`	Disconnect a connector's servers, leaving it enabled.
`/connectors auth <id>`	Manage a connector's credential (token, client secret) and report its auth state. Secrets live in the encrypted credential store; the manifest stores only the credential *name*.
`/connectors test <id>`	Connect in isolation with a throwaway probe, invoke one tool, and report per-step `[ ok ]`/`[fail]` results. Never touches the live session.
`/connectors stores [--check]`	Report each catalogue endpoint and its provenance (`default` or `config`); `--check` also probes each endpoint.
`/connectors marketplace show`	List the marketplaces configured for the Claude catalogue (the base endpoint plus each added marketplace, tagged `default`/`config`/`added`). A pure config read.
`/connectors marketplace claude list|add <source> [--force]|remove <name>`	List one catalogue's configured marketplaces, add an additional marketplace (an `owner/repo` slug, a `git@github.com:owner/repo.git` SSH remote, an `https://` marketplace-index URL, or a local directory), or remove one by name. Written to the user-global `connectors.stores.marketplaces`.
`/connectors help`	Print the usage block. A bare `/connectors` or an unknown subcommand does the same.

Sources accepted by `/connectors add`

- a catalogue connector id;
- a local directory holding a connector manifest;
- a local `.zip` or `.tar.gz` package;
- an `https://` URL naming a `.zip`/`.tar.gz` package (non-`https` URLs are refused).

Examples

List everything with the productivity filter:

```
/connectors list --category productivity
```

Browse the catalogue interactively, filtered by query and category, then install and enable the result:

```
/connectors claude drive --category productivity
/connectors add google-drive
/connectors auth google-drive
/connectors enable google-drive
```

Test a connector without touching the live session:

```
/connectors test google-drive
```

Check where the catalogue endpoint comes from:

```
/connectors stores --check
```

Output

Every report is prefixed with a `From: /connectors <sub>` attribution line (FR-006). A bare `/connectors`, `/connectors help`, and an unrecognised subcommand all render the same usage block (FR-017). All report text is ASCII.

`test` renders one line per harness step:

```
[ ok ] resolve servers (1 ms)
[ ok ] connect (12 ms)
[ ok ] tool invoke (3 ms)
```

and ends by confirming the harness was torn down with the live session untouched.

Configuration

The `connectors` block in `ragent.json` controls the subsystem (`enabled`, `store_dir`, the catalogue `stores` endpoints and fetch budgets, and the non-secret `credentials` name mapping). With `connectors.enabled: false`, every subcommand other than `help` reports the subsystem is disabled and no discovery, catalogue fetch, or connection occurs. Connectors are discovered under `.ragent/connectors/` (project), falling back to `~/.config/ragent/connectors/` (user-global).

See `docs/howtos/config.md` §7.37 and `specs/connectors/SPEC.md`.

Related

- `docs/howtos/connectors.md` - the full connector-system manual
- `/mcp` - MCP server status, discovery, and enable/disable
- `/plugins` - the sandboxed Codex/Claude plugin system (a sibling catalogue)
- `ragent connectors <sub>` - CLI parity for the same subcommands
- `specs/connectors/SPEC.md` - the full connector-system specification